Probabilistic Relational Outcome Logic (pRelOL)

Implemented rules

A reading and pencil-proof companion to the Lean development

Mathematical baseline: `prototype-3`, commit `3542464`

Source paths refreshed for the reorganized repository

1 The objects being related

Scope. This handbook selects the core implemented rules. It is not an exhaustive API listing or a claim of completeness. The companion theorem index links the statements to their Lean proofs.

A state relation $R(a, b)$ talks about two individual states. An *outcome assertion* $P(x, y)$ talks about two whole collections of outcomes. Keep these levels separate throughout a proof.

Symbol	Meaning
$M\alpha$	Outcomes over states of type α. Sets and weighted functions are instances.
$0,\ x + y,\ \eta a$	Empty outcome, additive combination, and pure outcome.
$x \gg\!= k$	Run a state kernel $k : \alpha \to M\gamma$ from every outcome in x.
$P \Rightarrow Q$	$\forall x, y,\ P(x, y) \Rightarrow Q(x, y)$.
$P^{\mathsf{T}}(y, x)$	$P(x, y)$, with the two arguments exchanged.

The common kernel judgment is

$$\boxed{\{P\}\ k \sim l\ \{Q\} \iff \forall x, y,\ P(x, y) \Rightarrow Q(x \gg\!= k, y \gg\!= l).}$$

For arbitrary outcome transformers F, G, replace the final pair by $(F(x), G(y))$; Lean calls that judgment `quadSem`. For commands, use their semantic kernels.

Assertions used repeatedly below are

$$\begin{aligned} \mathsf{emp}(x, y) &\iff x = 0 \ \wedge\ y = 0, \\ \mathsf{exactly}(x_0, y_0)(x, y) &\iff x = x_0 \ \wedge\ y = y_0, \\ (P \oplus Q)(x, y) &\iff \exists x_1, x_2, y_1, y_2, \quad x = x_1 + x_2 \ \wedge\ y = y_1 + y_2 \\ &\qquad\qquad \wedge P(x_1, y_1) \wedge Q(x_2, y_2). \end{aligned}$$

Thus $\oplus$ decomposes both outcomes. It is different from logical disjunction, which simply asks whether $P(x, y)$ or $Q(x, y)$ holds. Pieces may have overlapping support; their weights still add.

How to use this packet. Read a rule, write out its quantifiers, try its proof on paper, and only then inspect Lean. Use `README.md` for the study schedule, `exercises.md` for exercises, and `theorem-index.md` for exact declaration names. Paths here are relative to `docs/study/`; source paths below are relative to `RelationalOutcomeLogic/`. Module paths have changed; Lean declaration namespaces are preserved.

Lean source: `Generic/Assertions.lean`

Lean source: `Generic/Judgment.lean`

2 Shared structural rules

Scope. Any `OutcomeModel`: an additive commutative monoid with the pure/bind monad laws, zero annihilation, and distributivity of bind over addition in both arguments. No idempotence or normalization is assumed.

Write $(k; k')(a) = k(a) \gg\!= k'$ and $(k + k')(a) = k(a) + k'(a)$.

$$\frac{}{\{P\}\ \eta \sim \eta\ \{P\}}\ \textsc{Skip} \qquad \frac{P' \Rightarrow P \quad \{P\}\ k \sim l\ \{Q\} \quad Q \Rightarrow Q'}{\{P'\}\ k \sim l\ \{Q'\}}\ \textsc{Consequence}$$

$$\frac{\{P\}\ k \sim l\ \{Q\} \qquad \{Q\}\ k' \sim l'\ \{S\}}{\{P\}\ k; k' \sim l; l'\ \{S\}}\ \textsc{Sequence} \qquad \frac{\{P\}\ k \sim l\ \{Q\}}{\{P^{\mathsf{T}}\}\ l \sim k\ \{Q^{\mathsf{T}}\}}\ \textsc{Swap}$$

$$\frac{\{P_1\}\ k \sim l\ \{Q_1\} \qquad \{P_2\}\ k \sim l\ \{Q_2\}}{\{P_1 \oplus P_2\}\ k \sim l\ \{Q_1 \oplus Q_2\}}\ \textsc{Split}$$

$$\frac{\{P\}\ k_1 \sim l_1\ \{Q_1\} \qquad \{P\}\ k_2 \sim l_2\ \{Q_2\}}{\{P\}\ k_1 + k_2 \sim l_1 + l_2\ \{Q_1 \oplus Q_2\}}\ \textsc{Additive-choice}$$

$$\frac{\{P_1\}\ k \sim l\ \{Q\} \qquad \{P_2\}\ k \sim l\ \{Q\}}{\{P_1 \vee P_2\}\ k \sim l\ \{Q\}}\ \textsc{Disjunction}$$

Empty outcomes and exact inputs.

$$\{\mathsf{emp}\}\ k \sim l\ \{\mathsf{emp}\}, \qquad \{P\}\ (\lambda a.\, 0) \sim (\lambda b.\, 0)\ \{\mathsf{emp}\}.$$

$$\{\mathsf{exactly}(x, y)\}\ k \sim l\ \{Q\} \iff Q(x \gg\!= k, y \gg\!= l).$$

The empty assertion is satisfiable at $(0, 0)$; it is different from the always-false assertion. An empty-output kernel has no terminating outcomes.

Why splitting works. If $x = x_1 + x_2$, bind yields $(x_1 \gg\!= k) + (x_2 \gg\!= k)$. These two actual output pieces witness the conclusion. For arbitrary transformers F, G, this rule needs their additivity as an explicit premise; it does not follow merely from their type.

Why choice has a different precondition. Both branch pairs start from the same x, y. Their outputs are then added. Over probability weights, adding two normalized kernels has mass two: this is a rule for raw weighted kernels, not a probabilistic command that secretly chooses a branch with probability 1/2.

Lean source: `Generic/Rules.lean`

Main names: `quad_seq`, `quad_split`, `quad_choice`.

3 Weights and exact couplings

Scope. A semiring K equipped with `SummationSemiring`. Multiplication need not commute. Positivity and absence of zero divisors are not premises of coupling composition.

An outcome is a function $\mu : \alpha \to K$. The chosen indexed sum defines

$$\eta a(x) = \begin{cases} 1 & x = a, \\ 0 & x \neq a, \end{cases} \qquad (\mu \gg\!= k)(z) = \sum_a \mu(a)\, k(a)(z), \qquad \mathsf{mass}(\mu) = \sum_a \mu(a).$$

Support means nonzero weight: $\mathsf{supp}(\mu) = \{a \mid \mu(a) \neq 0\}$.

A joint $J : \alpha \times \beta \to K$ is an exact R-coupling of μ, ν when

$$\forall a,\ \sum_b J(a,b) = \mu(a), \qquad \forall b,\ \sum_a J(a,b) = \nu(b), \qquad J(a,b) \neq 0 \Rightarrow R(a,b).$$

Write $\mathsf{Cpl}_R(\mu,\nu)$ for existence of such a joint and $\mathsf{couple}(R)(\mu,\nu)$ for the corresponding outcome assertion.

$$\frac{R(a,b)}{\mathsf{Cpl}_R(\eta a, \eta b)}\ \textsc{Pure} \qquad \frac{\mathsf{Cpl}_R(\mu_1,\nu_1) \qquad \mathsf{Cpl}_R(\mu_2,\nu_2)}{\mathsf{Cpl}_R(\mu_1+\mu_2, \nu_1+\nu_2)}\ \textsc{Add}$$

$$\frac{\mathsf{Cpl}_R(\mu,\nu)}{\mathsf{Cpl}_R(r\mu, r\nu)}\ \textsc{Scale} \qquad \frac{\mathsf{Cpl}_R(\mu,\nu) \qquad \forall a,b,\ R(a,b) \Rightarrow S(a,b)}{\mathsf{Cpl}_S(\mu,\nu)}\ \textsc{Weaken}$$

$$\frac{\mathsf{Cpl}_R(\mu,\nu) \qquad \forall a,b,\ R(a,b) \Rightarrow \mathsf{Cpl}_S(k(a), l(b))}{\mathsf{Cpl}_S(\mu \gg\!= k, \nu \gg\!= l)}\ \textsc{Bind}$$

Here $r\mu$ multiplies each weight on the *left*. A bind witness is built by choosing a continuation joint $J_{a,b}$ and summing $J(a,b)\, J_{a,b}(c,d)$ over (a,b), in that multiplication order.

Two consequences are especially useful:

$$\mathsf{Cpl}_R(\mu,\nu) \Rightarrow \mathsf{mass}(\mu) = \mathsf{mass}(\nu), \qquad \mathsf{Cpl}_{=}(\mu,\nu) \iff \mu = \nu.$$

For probabilities, exact couplings match termination mass as well as outcomes. A bad cross pair does not by itself refute existence of a suitable joint.

Lean source: `Algebra/Weighted/Coupling.lean`

Main names: `Coupled.bind`, `Coupled.mass_eq`, `coupled_eq_iff`.

4 The pRHL fragment and command rules

Scope. The bridge is generic for weighted kernels. The command rules here use `Prob.Cmd` with ENNReal weights. Every sampling constructor carries a proof of mass one at every input state.

Define the statewise judgment by

$$[\,R\,]\ k \sim l\ [\,S\,] \iff \forall a, b,\ R(a,b) \Rightarrow \mathsf{Cpl}_S(k(a), l(b)).$$

The proved semantic equivalence is

$$\boxed{[\,R\,]\ k \sim l\ [\,S\,] \iff \{\mathsf{couple}(R)\}\ k \sim l\ \{\mathsf{couple}(S)\}.}$$

Forward: compose an input coupling with the statewise joints. Backward: apply the outcome judgment to related pure inputs. This explains how the two interfaces share proofs. It is not a translation of an external procedure or module language.

$$\frac{\forall a, b,\ R(a,b) \Rightarrow S(f(a), g(b))}{[\,R\,]\ \mathsf{assign}(f) \sim \mathsf{assign}(g)\ [\,S\,]}\ \textsc{Assign}$$

$$\frac{\forall a, \mathsf{mass}(k(a)) = 1 \quad \forall b, \mathsf{mass}(l(b)) = 1 \qquad \forall a, b,\ R(a,b) \Rightarrow \mathsf{Cpl}_S(k(a), l(b))}{[\,R\,]\ \mathsf{sample}(k) \sim \mathsf{sample}(l)\ [\,S\,]}\ \textsc{Sample}$$

$$\frac{R \Rightarrow (g \leftrightarrow h) \qquad [\,R \wedge g\,]\ c_1 \sim d_1\ [\,S\,] \qquad [\,R \wedge \neg g\,]\ c_2 \sim d_2\ [\,S\,]}{[\,R\,]\ \mathsf{if}\ g\ \mathsf{then}\ c_1\ \mathsf{else}\ c_2 \sim \mathsf{if}\ h\ \mathsf{then}\ d_1\ \mathsf{else}\ d_2\ [\,S\,]}\ \textsc{If}$$

Here g tests the left state and h the right. Skip, consequence, sequence, symmetry, and input case analysis are also implemented.

One-sided steps. Assignment on the left against skip requires $R(a,b) \Rightarrow S(f(a), b)$. Left sampling against skip requires normalization and

$$R(a,b) \Rightarrow \forall t \in \mathsf{supp}(k(a)),\ S(t,b).$$

A left conditional against any fixed right command d is proved with two premises, under $R \wedge g$ and $R \wedge \neg g$; it needs no right guard. Right-sided counterparts follow by symmetry.

Quantitative assertion rule. If $\{P\}\ c \sim d\ \{Q\}$ then $\{r \cdot P\}\ c \sim d\ \{r \cdot Q\}$, where $(r \cdot P)(\mu, \nu) \iff \exists \mu_0, \nu_0,\ \mu = r\mu_0 \wedge \nu = r\nu_0 \wedge P(\mu_0, \nu_0)$. This is an image assertion, not division by r.

Lean source: `Generic/Coupling.lean`

Lean source: `Subsumption/PRHL/Embedding.lean`

Lean source: `Subsumption/PRHL/OneSided.lean`

Lean source: `Prob/Logic/Rules.lean`

5 Loop semantics and synchronous reasoning

Scope. Generic iteration uses `IterationModel` and complete lattices of outcomes. Weighted loop couplings use `OmegaSummationSemiring`. One-sided losslessness below uses probability.

The finite approximants count loop unfoldings, starting with no terminating output:

$$\begin{aligned} L_0(s) &= 0, \\ L_{n+1}(s) &= \begin{cases} k(s) \gg\!= L_n & g(s), \\ \eta s & \neg g(s), \end{cases} \\ \mathsf{Loop}(g,k)(s) &= \sup_n L_n(s). \end{aligned}$$

The sequence is increasing. Bind preserves its increasing limit, yielding both the unfolding equation and the least-fixed-point characterization:

$$\mathsf{Loop}(g,k)(s) = \begin{cases} k(s) \gg\!= \mathsf{Loop}(g,k) & g(s), \\ \eta s & \neg g(s). \end{cases}$$

An always-true loop has empty terminating semantics. Probability commands have termination mass at most one; missing mass represents nontermination.

$$\frac{R \Rightarrow (g \leftrightarrow h) \qquad [\,R \wedge g\,]\ k \sim l\ [\,R\,]}{[\,R\,]\ \mathsf{Loop}(g,k) \sim \mathsf{Loop}(h,l)\ [\,R \wedge \neg g \wedge \neg h\,]}\ \textsc{While-sync}$$

There is no separate termination premise. The proof builds one product execution whose increasing joint approximants have the required marginals. Arbitrarily chosen finite couplings need not form an increasing sequence, so they cannot simply be passed to the limit. The outcome-level version replaces brackets by coupling assertions using the pRHL bridge.

A loop against skip needs more. Let

$$\mathsf{LosslessOn}(A,c) \iff \forall a,\ A(a) \Rightarrow \mathsf{mass}(\mathsf{sem}(c)(a)) = 1.$$

$$\frac{[\,R \wedge g\,]\ c \sim \mathsf{skip}\ [\,R\,] \qquad \mathsf{LosslessOn}(\lambda a.\exists b,\ R(a,b), \mathsf{while}\ g\ \mathsf{do}\ c)}{[\,R\,]\ \mathsf{while}\ g\ \mathsf{do}\ c \sim \mathsf{skip}\ [\,R \wedge \neg g\,]}\ \textsc{While-left}$$

This premise concerns the *whole loop*. A lossless body is insufficient: $\mathsf{while\ true\ do\ skip}$ has mass zero, whereas skip has mass one. Exact marginals cannot relate them. The right-sided theorem is symmetric.

Lean source: `Generic/Loops/Semantics.lean`

Lean source: `Generic/Loops/Synchronous.lean`

Lean source: `Subsumption/PRHL/OneSided.lean`

6 Ranked asynchronous loops

Scope. Ordered summation semirings; probability and Set interfaces also exist. The schedule is supplied by the proof author. No automatic invariant or scheduler discovery is implemented.

Two loops need not advance together. A scheduler chooses a mode for each state pair $p = (a, b)$ and a rank $\rho(p) \in \mathbb{N}$. All obligations below are required whenever the invariant $R(a, b)$ holds and that mode is selected.

Mode	Guard obligation	Required local coupling
Left	$g(a)$	Couple $k(a)$ with ηb, supported by R and strict rank decrease.
Right	$h(b)$	Couple ηa with $l(b)$, supported by R and strict rank decrease.
Both	$g(a) \wedge h(b)$	Couple $k(a)$ with $l(b)$, supported by R.
Done	$\neg g(a) \wedge \neg h(b)$	Exit; no body step.

More precisely, for a unilateral step from p, the coupling relation is

$$R_{<p}(x, y) \iff R(x, y) \wedge \rho(x, y) < \rho(p).$$

The left obligation is $\mathsf{Cpl}_{R_{<p}}(k(a), \eta b)$; the right is $\mathsf{Cpl}_{R_{<p}}(\eta a, l(b))$. The both obligation is $\mathsf{Cpl}_{R}(k(a), l(b))$.

$$\frac{\text{A schedule satisfying the four obligations above}}{[\,R\,]\ \mathsf{Loop}(g, k) \sim \mathsf{Loop}(h, l)\ [\,R \wedge \neg g \wedge \neg h\,]}\ \textsc{While-async}$$

What the theorem proves. The chosen local joints define a product loop. Its full output joint has the *original two loops* as exact marginals. These marginal equations are a theorem, not extra promises hidden in the schedule.

What the rank ensures. Every supported unilateral successor lowers the natural rank. Consequently an uninterrupted unilateral phase cannot continue forever. Both-mode steps may reset the rank, and infinitely many joint steps may still diverge. There is no uniform global bound on the number of loop steps.

Coupling a unilateral body with a pure state also forces that body's total weight to be one. In probability this is losslessness of the selected step.

Boundary to remember. Almost-sure termination of an unbounded random retry phase does not supply this strict support-wise rank. The existing mixed rule does not cover such a phase merely from its probability-one termination proof. The separate lossless loop-versus-skip rule on the previous page can handle cases outside this scheduler's premises.

Lean source: `Generic/Loops/Asynchronous.lean`

Main names: `AsyncSchedule`, `AsyncSchedule.isCoupling_loop`, `prhl_while_async`, `quad_couple_while_async`.

7 Quantitative refutation certificates

Scope. Probability weights, or the generic ordered summation-semiring instance. Only the selected events must be finite; whole output distributions need not have finite support.

For finite $A \subseteq \alpha$ and $B \subseteq \beta$, write $\mu(A) = \sum_{a \in A} \mu(a)$. Define the cover condition by

$$\mathsf{Cover}(R, A, B) \iff \forall a \in A,\ \forall b,\ R(a, b) \Rightarrow b \in B.$$

The quantifier over b is global, including states outside the support of ν. Every exact coupling obeys the event bound

$$\mathsf{Cpl}_R(\mu, \nu) \ \wedge\ \mathsf{Cover}(R, A, B) \quad \Rightarrow \quad \mu(A) \leq \nu(B).$$

Hence a `MassDeficitCert` stores A, B, their cover proof, and a strict gap:

$$\frac{\mathsf{Cover}(R, A, B) \qquad \nu(B) < \mu(A)}{\neg \mathsf{Cpl}_R(\mu, \nu)}\ \textsc{Deficit}$$

$$\frac{\mathsf{Cover}(R, A, B) \qquad L \leq \mu(A) \quad \nu(B) \leq U \quad U < L}{\neg \mathsf{Cpl}_R(\mu, \nu)}\ \textsc{Bounds}$$

Two lower bounds cannot establish a deficit. In particular, finite loop unrolling usually gives a lower bound on terminating outcomes, not the right-side upper bound required here. No subtraction or division is used in the certificate.

Refuting the actual contract. Let F, G be the two actual outcome transformers. A violating pair of permitted inputs refutes the judgment:

$$\frac{P(x, y) \qquad Q \Rightarrow \mathsf{couple}(R) \qquad \neg \mathsf{Cpl}_R(F(x), G(y))}{\neg \{P\}\ F \sim G\ \{Q\}}\ \textsc{Refute}$$

Here the displayed judgment is the transformer version, `quadSem`. The program certificate `BugCert` fixes states a_0, b_0 and a deficit on $F(\eta a_0), G(\eta b_0)$; its use still requires the original precondition at those pure inputs. A failure to discover a joint is not a refutation theorem.

Concrete calculation. Under equality, take $A = B = \{\mathsf{false}\}$. If one output assigns mass $1/2$ to false and the other assigns $1/4$, the cover is immediate and the strict gap refutes equality coupling. Matching supports do not repair the mismatch. This is the shape of the cached-randomness bug.

Lean source: `Prob/Refutation/Certificate.lean`

Lean source: `Algebra/Weighted/Events.lean`

Lean source: `Generic/Refutation.lean`

Main names: `MassDeficitCert.refutes`, `MassDeficitCert.of_bounds`, `BugCert.refutes_of_entails`.

8 Observations, splitting, and the search contract

Scope. The rich observation and mass-labelled splitting APIs here are implemented in the probability layer. Restrictions keep the original weights; they do not renormalize.

An observation $f : \alpha \to \gamma$ maps outputs by $f_*\mu = \mu \gg\!= (\lambda a.\eta(f(a)))$. If $R(a,b) \Rightarrow S(f(a), g(b))$, then

$$\mathsf{Cpl}_R(\mu,\nu) \Rightarrow \mathsf{Cpl}_S(f_*\mu, g_*\nu).$$

A deficit on these *actual observed outputs* refutes an internal contract Q when $Q(\mu,\nu) \Rightarrow \mathsf{Cpl}_S(f_*\mu, g_*\nu)$. One must also prove the certificate input satisfies the original precondition.

Write $\mathsf{massEq}(p,q)(\mu,\nu)$ for $\mathsf{mass}(\mu) = p \wedge \mathsf{mass}(\nu) = q$. **Split one joint by a shared event.** Suppose $\mathsf{Cpl}_R(\mu,\nu)$ and $R(a,b) \Rightarrow (E(a) \leftrightarrow D(b))$. Write $\mu|_E$ for restriction, $p = \mathsf{mass}(\mu|_E)$, and $q = \mathsf{mass}(\mu|_{\neg E})$. Then

$$\Big((\mathsf{couple}(R \wedge E \wedge D) \wedge \mathsf{massEq}(p,p)) \;\oplus\; (\mathsf{couple}(R \wedge \neg E \wedge \neg D) \wedge \mathsf{massEq}(q,q))\Big)(\mu,\nu).$$

The two pieces come from restricting the *same joint*. Their marginals are exactly the corresponding restrictions, even when one piece has mass zero.

The implemented end-to-end example. A trial has miss weight q and hit weight r, with $q + r = 1$. Both inputs are pure (n, false): budget n and no hit yet. Full scan performs all trials; early exit stops at the first hit. Define

$$S_0 = 0, \qquad S_{n+1} = qS_n + r, \qquad q^n + S_n = 1.$$

The contract is a split into the following two outcome classes:

Piece	Mass on each side	Coupled relation
Failure	q^n	Both answers are false; both remaining budgets are zero; each used n trials.
Success	S_n	Both answers are true; early exit uses at most as many trials as full scan; full scan used n.

`search_outcome_quad` proves this contract using asynchronous alignment and joint splitting. For $n \geq 2$ and $r > 0$, strict trial savings have positive probability. These are consumed-trial results, not an expected-runtime theorem.

The faulty cached implementation samples once and reuses the answer. For $0 < q < 1$ and $n > 1$, its failure probability is $q > q^n$. Observe only the answer flag, place cached output on the deficit's left, and transfer the contradiction back to the original contract. `cached_outcome_spec_refuted` refutes precisely the contract proved for early exit.

Lean source: `Prob/Coupling/Observation.lean`

Lean source: `Prob/Refutation/Observation.lean`

Lean source: `Prob/Coupling/Restriction.lean`

Lean source: `Examples/Probability/Search/Case.lean`

9 The Set layer and the family loop rule

Scope. Set outcomes use union as addition. This page records Set-specific APIs; the general weighted outcome-assertion family loop rule has not been implemented.

For sets X, Y, distinguish the following assertions:

$$\begin{aligned}
\mathsf{atomW}(R)(X,Y) &\iff \forall a \in X,\ \forall b \in Y,\ R(a,b),\\
\mathsf{atom}(R)(X,Y) &\iff X \neq \varnothing \wedge Y \neq \varnothing \wedge \mathsf{atomW}(R)(X,Y),\\
\mathsf{coupleL}(R)(X,Y) &\iff \forall a \in X,\ \exists b \in Y,\ R(a,b),\\
\mathsf{couple}(R)(X,Y) &\iff \mathsf{coupleL}(R)(X,Y) \wedge \forall b \in Y,\ \exists a \in X,\ R(a,b).
\end{aligned}$$

Bitotal coupling has a witness relation whose two projections are exactly X, Y. It can hold for two empty sets. Adding nonemptiness gives $\mathsf{coupleNE}(R)$, so

$$(\mathsf{coupleNE}(R) \oplus \top)(X,Y) \iff \exists a \in X,\ \exists b \in Y,\ R(a,b).$$

Without nonemptiness, $\mathsf{couple}(R) \oplus \top$ is always true by selecting an empty coupling piece. Refuting a product atom may use a bad reachable cross pair; refuting a coupling needs to rule out possible partners, not just one pairing.

Family loop rule. Let ϕ_n describe active outcomes after n body steps and ψ_n outcomes that exit then. Require:

1. $\phi_n(X,Y)$ forces every state of X to satisfy g and every state of Y to satisfy h.
2. $\psi_n(X,Y)$ forces every state of X to satisfy $\neg g$ and every state of Y to satisfy $\neg h$.
3. For every n, $\{\phi_n\}\ c \sim d\ \{\phi_{n+1} \oplus \psi_{n+1}\}$.
4. For *every* family E_n, E'_n, if $\psi_n(E_n, E'_n)$ for all n, then $\psi_\infty(\bigcup_n E_n, \bigcup_n E'_n)$. Lean calls this `Converges`.

Then

$$\boxed{\{\phi_0 \oplus \psi_0\}\ \mathsf{while}\ g\ \mathsf{do}\ c \sim \mathsf{while}\ h\ \mathsf{do}\ d\ \{\psi_\infty\}.}$$

There is no termination premise. All terminating exits are collected by the union. The convergence premise quantifies over arbitrary families satisfying ψ_n; it does not secretly assume these families are increasing.

One-sided choice. From $\{P\}\ c_1 \sim d\ \{Q_1\}$ and $\{P\}\ c_2 \sim d\ \{Q_2\}$, Set RelOL derives $\{P\}\ c_1 + c_2 \sim d\ \{Q_1 \oplus Q_2\}$. This duplicates the passive outcome using $Y \cup Y = Y$. The same argument fails for non-idempotent probability addition.

Lean source: `Set/Assertions.lean`

Lean source: `Set/Coupling.lean`

Lean source: `Set/Rules.lean`

10 Shared assumptions and the five scalar models

Scope. The scalar construction uses total summation semirings. A bare semiring provides neither arbitrary indexed sums nor the order and continuity needed for full loops.

The assumptions have separate jobs:

Interface	What it provides
`OutcomeModel`	Empty, addition, pure, bind; monoid, monad, and distributive laws. Shared structural rules.
`SummationSemiring`	Chosen arbitrary indexed sums with singleton, addition, exchange, product-index, and left/right multiplication laws. Constructs weighted bind and couplings.
`PositiveSummation`	A sum is zero iff all summands are zero. With no zero divisors, supplies exact support descriptions.
`IterationModel`	Complete lattices of outcomes, empty as bottom, monotone bind, and preservation of increasing countable limits in the continuation.
`OmegaSummationSemiring`	Complete scalar lattice with zero as bottom, monotone operations, and increasing-limit laws for sums and multiplication. Derives weighted iteration.

All five implemented scalar models satisfy the required summation and increasing limit laws, as well as positivity and absence of zero divisors.

Model	$+$	$\cdot$	$0, 1$	Meaning
Reachability	$\vee$	$\wedge$	false, true	Possible output
$\mathbb{R}^{\infty}_{\geq 0}$	$+$	$\times$	$0, 1$	Nonnegative weight
$\mathbb{N}^{\infty}$	$+$	$\times$	$0, 1$	Trace count
Tropical cost	min	$+$	$\infty, 0$	Cheapest cost
Word languages	$\cup$	concat.	$\varnothing, \{\epsilon\}$	Emitted finite words

For tropical costs the semantic order reverses numerical order, so semantic suprema become numerical infima. Language multiplication is noncommutative; the order of emissions matters. Languages may contain infinitely many finite words. ENNReal weights become probabilities only when the appropriate mass constraints are established.

One calculation, several meanings. A direct route of weight d or two successive edges of weights a, b gives output weight $d + ab$. Boolean weights record reachability; unit counting weights give two traces; costs $7, 2, 3$ give minimum cost 5. A weighted countdown gives $r^n \eta 0$ in all instances.

Scope. This is not every partial algebra from Outcome Logic: operations are total, weights have no countable-support field, iteration uses increasing sequential limits, and trace words are finite. Compatibility theorems connect the generic construction to the existing Set and ENNReal developments.

Lean sources: `Algebra/Scalar/Basic.lean`, `Algebra/Scalar/Order.lean`, `Bridges/Set.lean`, `Bridges/Probability.lean`, `Examples/Algebra/Routes.lean`.

11 A checkpoint before further development

Scope. Use this page to test understanding after the exercises. The aim is to reconstruct why a proof is valid, not to memorize theorem names or read every file in order.

Reconstruct these arguments without opening their proofs.

1. **Sequence and split.** Explain exactly where associativity and additivity of bind enter. Give the witness decomposition for split.
2. **The pRHL bridge.** Prove both directions. Identify where related pure inputs are used and where arbitrary coupled inputs are used.
3. **Coupling versus product atoms.** Give a finite example where an equality coupling exists although a cross pair fails equality.
4. **Deficit certificate.** Calculate the two-coin event masses, prove global cover, and state the original precondition and postcondition implication needed to refute a program judgment.
5. **Synchronous limits.** Explain why a coherent increasing sequence of joints is sufficient and unrelated finite witnesses are insufficient.
6. **Asynchronous alignment.** Draw a schedule for a machine with a bookkeeping phase. Show the guard and rank obligation in each mode.
7. **Search.** Derive the budget-two result by hand, then explain the arbitrary-budget proof using alignment, restriction, and a mass recurrence.
8. **Generalization.** Reinterpret one weighted calculation as reachability, counting, cost, and a trace language. Identify the scalar laws used.

What this checkpoint already contains.

- Set and weighted outcome assertions, exact couplings, and refutation.
- Semantic exact-coupling pRHL equivalence and core command rules.
- Full loop semantics, synchronous rules, lossless one-sided rules, and ranked mixed asynchronous alignment.
- The full-scan/early-exit contract and a cached-randomness refutation of that same contract; shared algebra with five scalar instances and examples.

What remains a future direction. Informed symbolic execution and coupling/certificate discovery, richer case studies, a general weighted outcome-family loop rule, broader asynchronous progress criteria, and external LeanCrypt integration. The current Set `symexec` is a simplification wrapper. These are not needed to complete the study checkpoint. Completeness is not a goal.

Reading question. When comparing Outcome Logic, pRHL/product programs, and Quantitative Weakest Hyper Pre, distinguish semantic expressiveness from local proof rules, reusable invariants, automation, and empirical proof effort. This development does not establish a new expressiveness separation from QWHP.

Ready to resume? Explain the search correctness and bug proofs from the definitions, reconstruct several small proofs independently, and list the exact assumptions of the next rule or tactic you want. Use the companion study plan for selected paper readings and a place to record unresolved questions.